CS61C

Great Ideas in Computer Architecture
(a.k.a. Machine Structures)

Assistant Teaching Professor
Lisa Yan

RISC-V Datapath I

Great Idea #1: Abstraction (Levels of Representation/Interpretation)

High Level Language Program (e.g., C)

Compiler

Assembly Language Program (e.g., RISC-V)

Assembler

Machine Language Program (RISC-V)

Hardware Architecture Description (e.g., block diagrams)

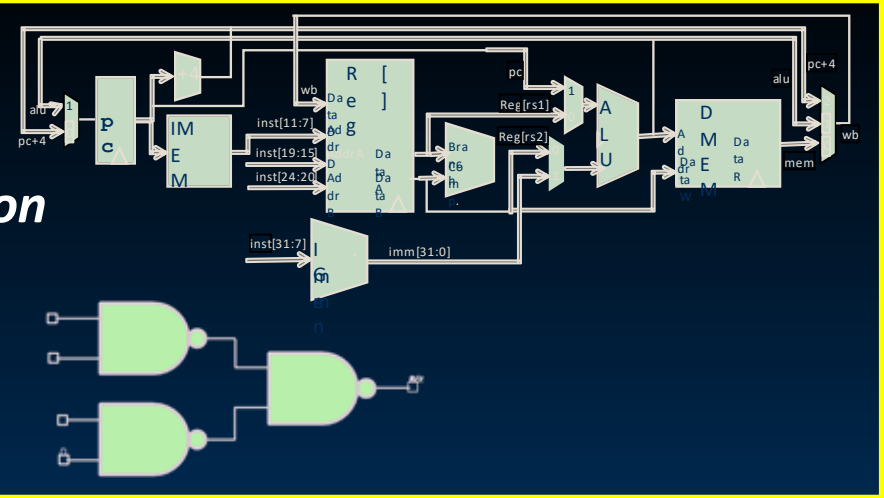
Architecture Implementation

Logic Circuit Description (Circuit Schematic Diagrams)

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

```
lw      x3, 0(x10)
lw      x4, 4(x10)
sw      x4, 0(x10)
sw      x3, 4(x10)
```

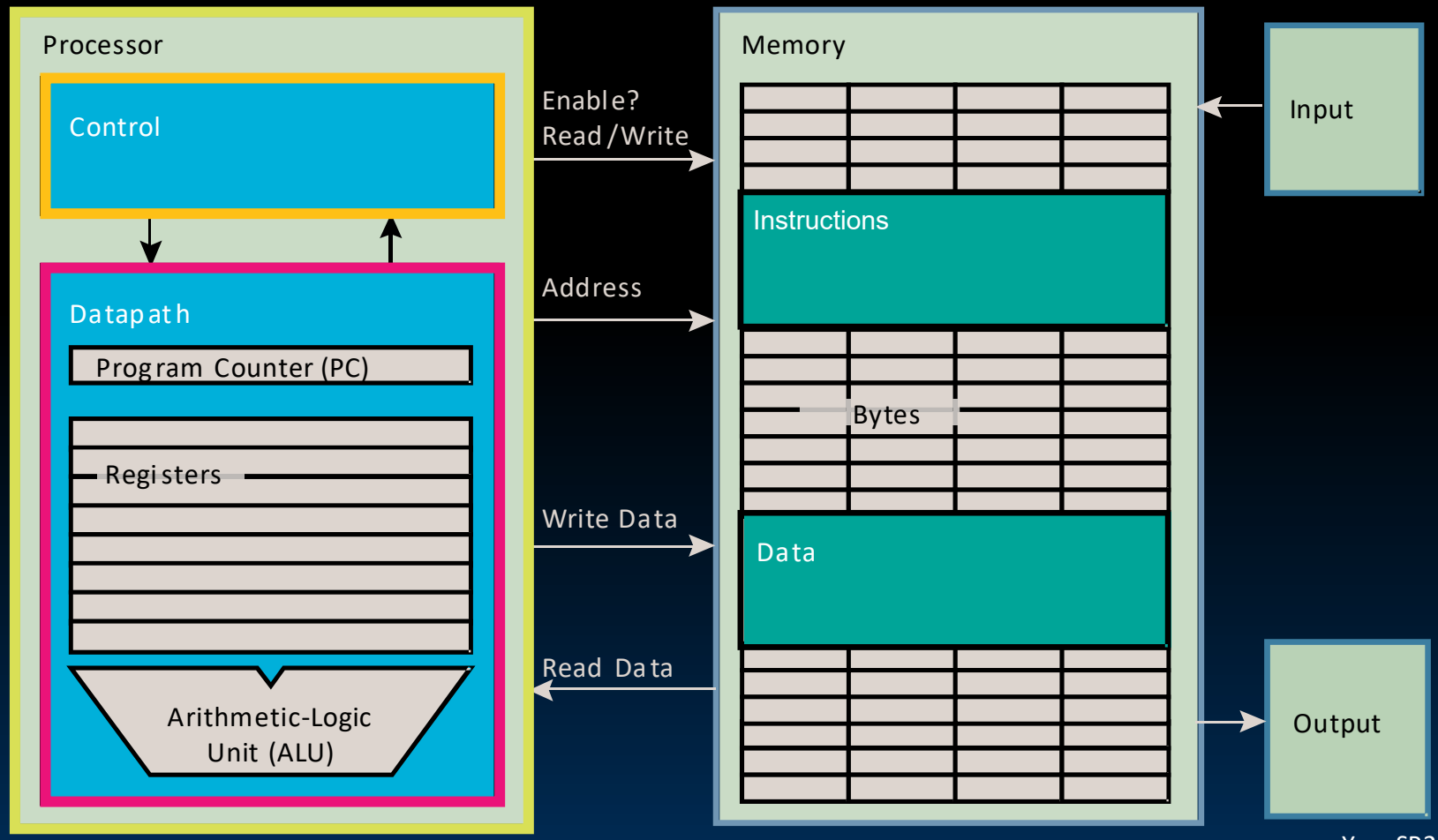
```
1000 1101 1110 0010 0000 0000 0000 0000
1000 1110 0001 0000 0000 0000 0000 0100
1010 1110 0001 0010 0000 0000 0000 0000
1010 1101 1110 0010 0000 0000 0000 0100
```



How do we build a Single-Core Processor?

Processor (CPU): the active part of the computer that does all the work (data manipulation and decision-making).

- *Datapath* ("the brawn"): portion of the processor that contains hardware necessary to perform operations required by the processor.
- *Control* ("the brain"): portion of the processor (also in hardware) that tells the datapath what needs to be done.



5 Steps to a RISC-V Instruction

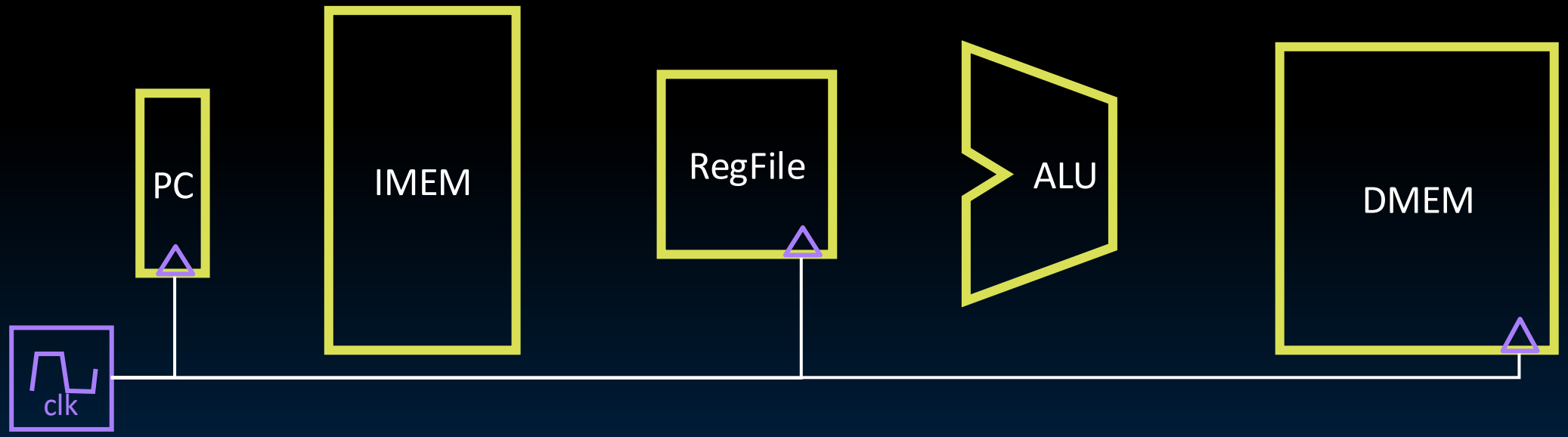
- 5 Steps to a RISC-V Instruction
- R-Type: **`add`** datapath
- R-Type: **`sub`** datapath
- I-Type: **`addi`** datapath
- Supporting Loads and Stores
- Designing the Immediate Generation Block
- [Reference] CPU Elements

What Key CPU Elements Do We Need?

In each clock cycle, RV32I instruction data passes through these elements.

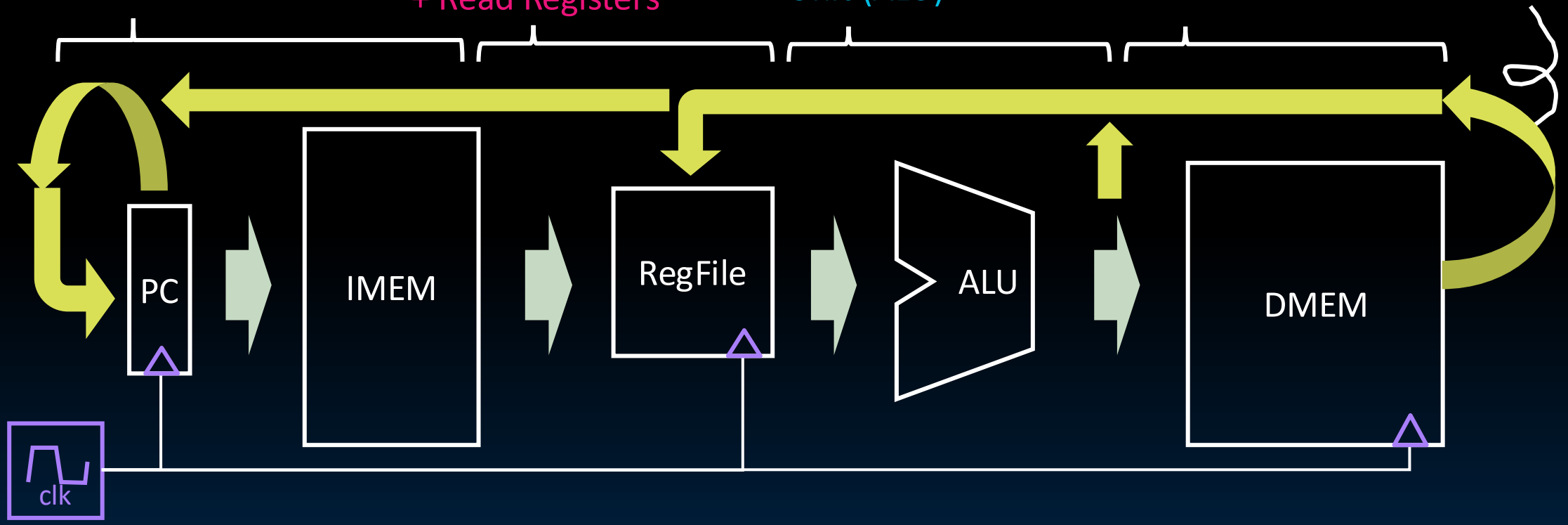
- ALU, Read State: Combinational logic with delay
- Write state: **rising-edge triggered** with clock

Read the notes for details!



5 Basic Steps ("Stages") of Instruction Execution

1. Instruction Fetch (**IF**)
2. Instruction Decode (**ID**) + Read Registers
3. Execute (**EX**) Arithmetic Logic Unit (ALU)
4. Memory Access (**MEM**)
5. Write back to Register (**WB**)



- Modularity: **Break up the process into steps**
  - Easy to optimize one stage without touching the others

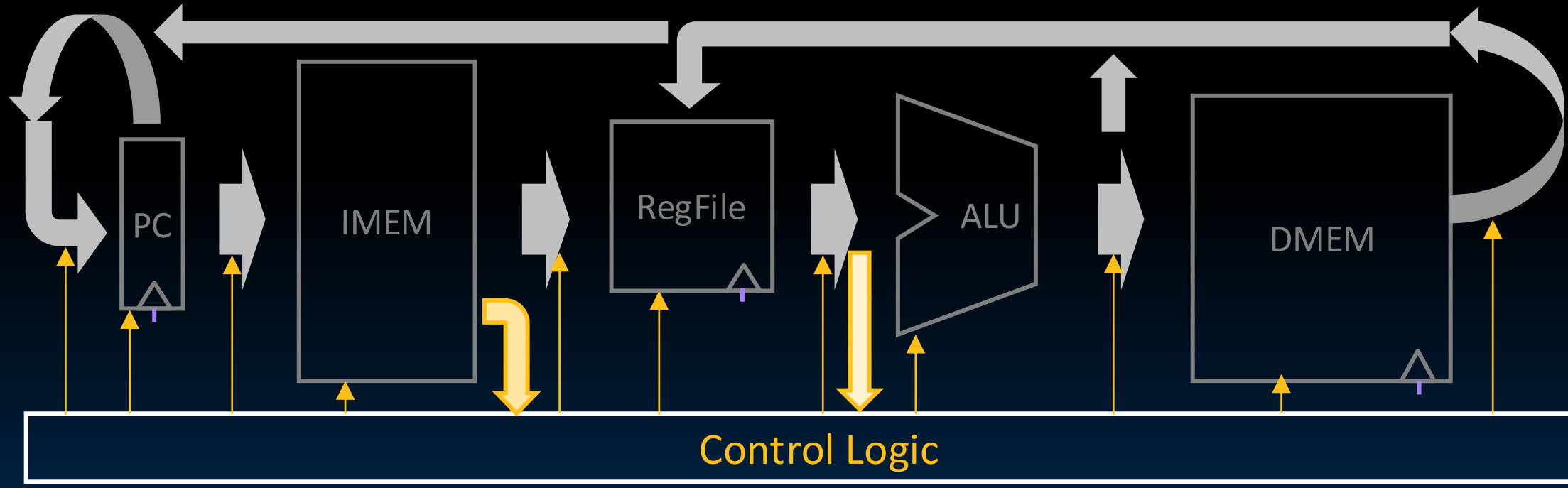
Datapath vs. Control

Space Station (Datapath)

Mission Control (Control Logic)

The control logic selects "needed" datapath lines based on the instruction.

- MUX selector, ALU op selector, write enable, etc.
- Control Logic specifies which instruction to execute.



Control and Datapath are two *abstractions* that together comprise the CPU. Under the hood, both are designed with combinational logic and/or state elements.

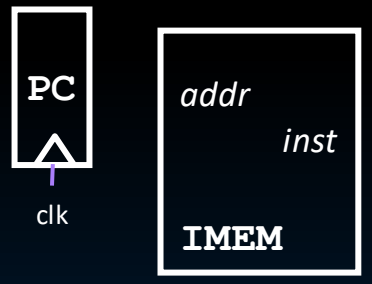
Our Plan: Iteratively Build Processor

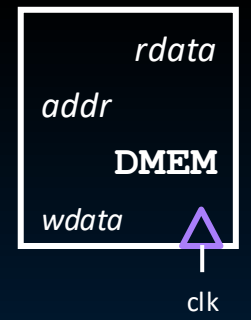
In each clock cycle, RV32I instruction data passes through these elements.

- ALU, Read State: Combinational logic with delay
- Write state: **rising-edge triggered** with clock

Read the notes for details!

Which elements are "clocked," i.e., synchronous state elements?





Control Logic

Your Own Design

1. We should use the main ALU to compute `PC=PC+4` in order to save some gates.
2. The ALU is a synchronous state element.
3. The program counter (PC) is a register.

FFF 0%

FFT 0%

FTF 0%

FTT 0%

TFF 0%

TFT 0%

TTF 0%

TTT 0%

Your Own Design

1. We should use the main ALU to compute `PC=PC+4` in order to save some gates.
2. The ALU is a synchronous state element.
3. The program counter (PC) is a register.

	123
A.	FFF
B.	FFT
C.	FTF
D.	FTT
E.	TFF
F.	TFT
G.	TTF
H.	TTT

*synchronous state element are clocked.

Not all state elements are clocked; some like IMEM operate like combinational logic!

R-Type: `add` Datapath

- 5 Steps to a RISC-V Instruction
- R-Type: **`add`** datapath
- R-Type: **`sub`** datapath
- I-Type: **`addi`** datapath
- Supporting Loads and Stores
- Designing the Immediate Generation Block
- [Reference] CPU Elements

Implementing the add Instruction

- **Suppose we had a single instruction in our RISC-V ISA: `add`.**

```
add  rd rs1 rs2
```

Example add-only program

```
0x100 | add x18 x18 x10
0x104 | add x18 x18 x18
  ... | add ...
```

- **The `add` instruction makes two changes to processor state:**
  - RegFile
  - PC

```
R[rd] = R[rs1] + R[rs2]
PC = PC + 4
```

Our Plan: Iteratively Build Processor

For now, we will disconnect memory.
By the end of next lecture, we'll build the datapath that supports the full RV32I ISA.

Control Logic

Datapath for add

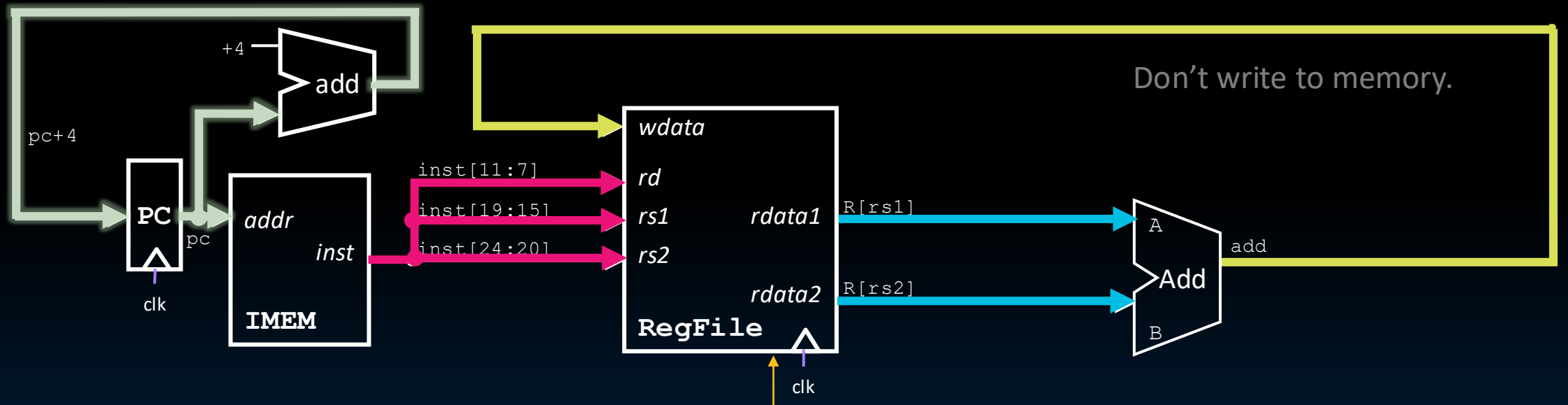
```
PC = PC + 4          R[rd] = R[rs1] + R[rs2]
```

Increment PC to next instruction.

Split instruction to index into RegFile.

Feed read register values into Add.

Write Add output to destination register.



RegWEn
1

Control Logic

R-Type: `sub` Datapath

- 5 Steps to a RISC-V Instruction
- R-Type: **`add`** datapath
- R-Type: **`sub`** datapath
- I-Type: **`addi`** datapath
- Supporting Loads and Stores
- Designing the Immediate Generation Block
- [Reference] CPU Elements

Implementing the sub Instruction

- What if our ISA now had two instructions: add/sub?

```
sub   rd rs1 rs2
```

- sub is almost the same as add, except now the ALU subtracts:
  - RegFile `R[rd] = R[rs1] - R[rs2]`
  - PC `PC = PC + 4`
- *Control logic* changes!

Datapath for sub

31 25	24 20	19 15	14 12	11 7	6 0
0100000	rs2	rs1	funct3	rd	0110011

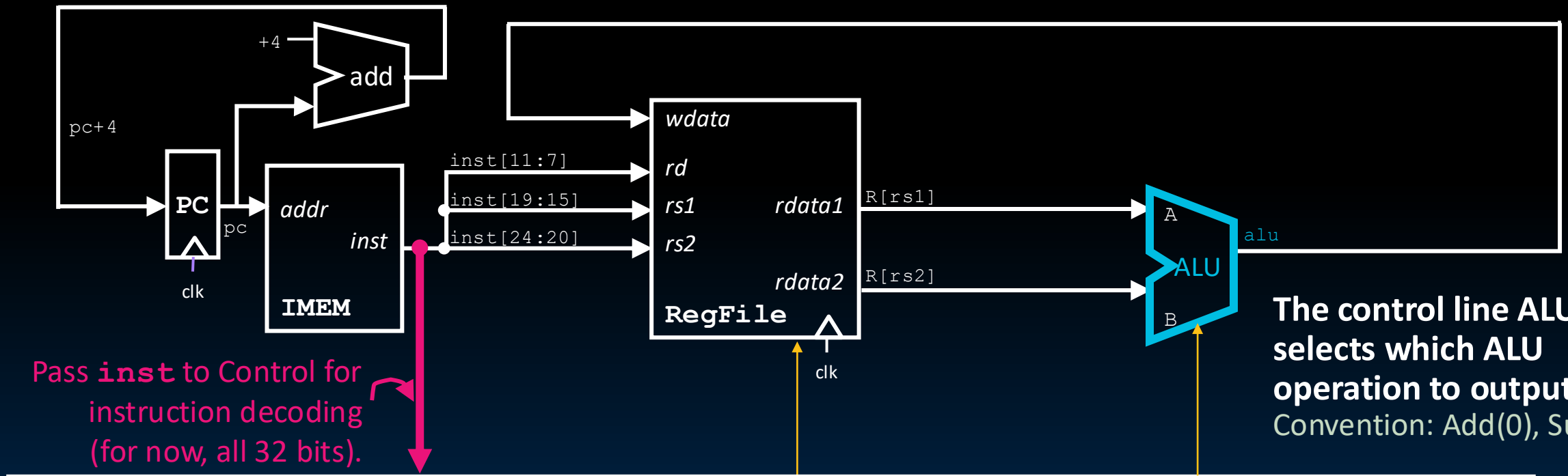
`PC = PC + 4` `R[rd] = R[rs1] - R[rs2]`

Increment PC to next instruction.

Split instruction to index into RegFile.

Feed read register values *into ALU.*

Write *ALU output* to destination register.



The control line ALUSel selects which ALU operation to output.
Convention: Add(0), Sub(1).

Supporting all R-Type Instructions

funct7			funct3		opcode	
0000000	rs2	rs1	000	rd	0110011	add
0100000	rs2	rs1	000	rd	0110011	sub
0000000	rs2	rs1	001	rd	0110011	sll
0000000	rs2	rs1	010	rd	0110011	slt
0000000	rs2	rs1	011	rd	0110011	sltu
0000000	rs2	rs1	100	rd	0110011	xor
0000000	rs2	rs1	101	rd	0110011	srl
0100000	rs2	rs1	101	rd	0110011	sra
0000000	rs2	rs1	110	rd	0110011	or
0000000	rs2	rs1	111	rd	0110011	and

ALUSel
add,sub,xor,and,or,slt,sltu,sll,sra,srl

The Control Logic decodes **funct3**, **funct7** instruction fields and selects appropriate ALU function by setting the control line ALUSel.

I-Type: `addi` Datapath

- 5 Steps to a RISC-V Instruction
- R-Type: **`add`** datapath
- R-Type: **`sub`** datapath
- I-Type: **`addi`** datapath
- Supporting Loads and Stores
- Designing the Immediate Generation Block
- [Reference] CPU Elements

Implementing the addi Instruction

- Suppose we add a new instruction: addi, RV32I I-Type:

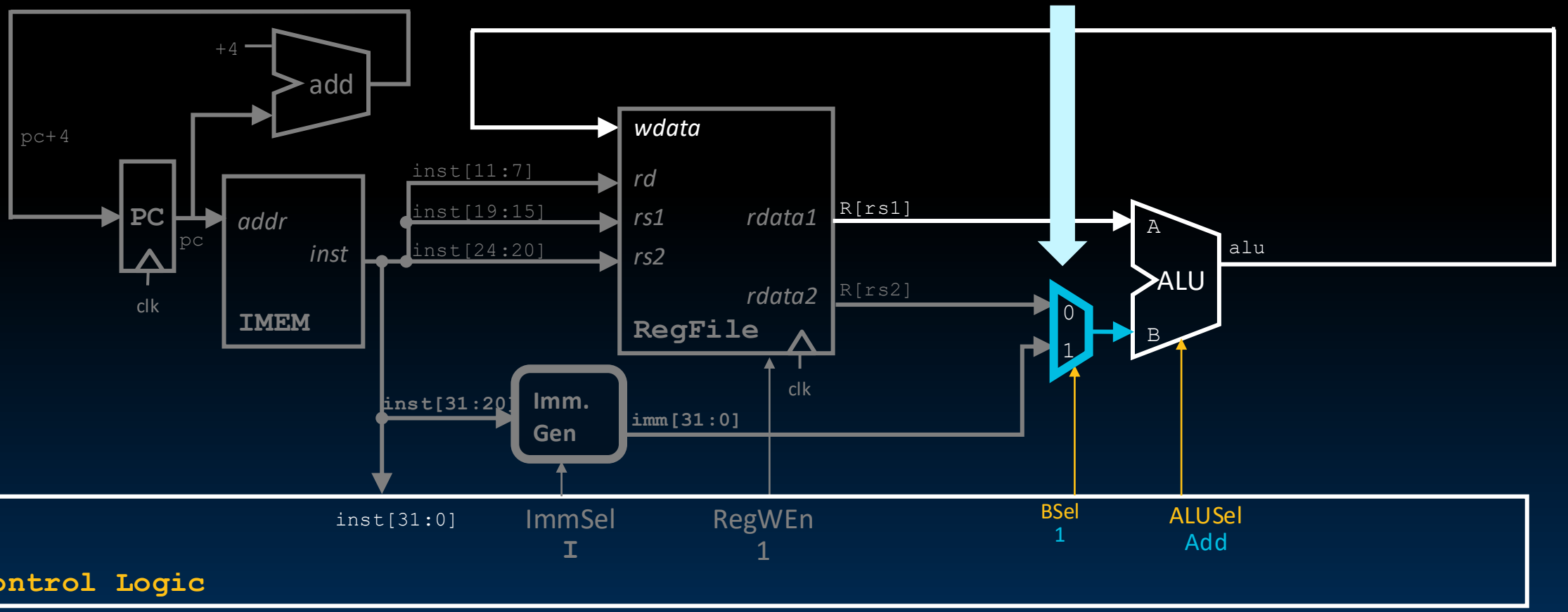
```
addi   rd rs1 imm
```

- The addi instruction updates the same two states as before. But we now need to build an immediate imm!
  - RegFile
  - PC

```
R[rd] = R[rs1] + imm
PC = PC + 4
```

1. New MUX to Select Immediate for ALU

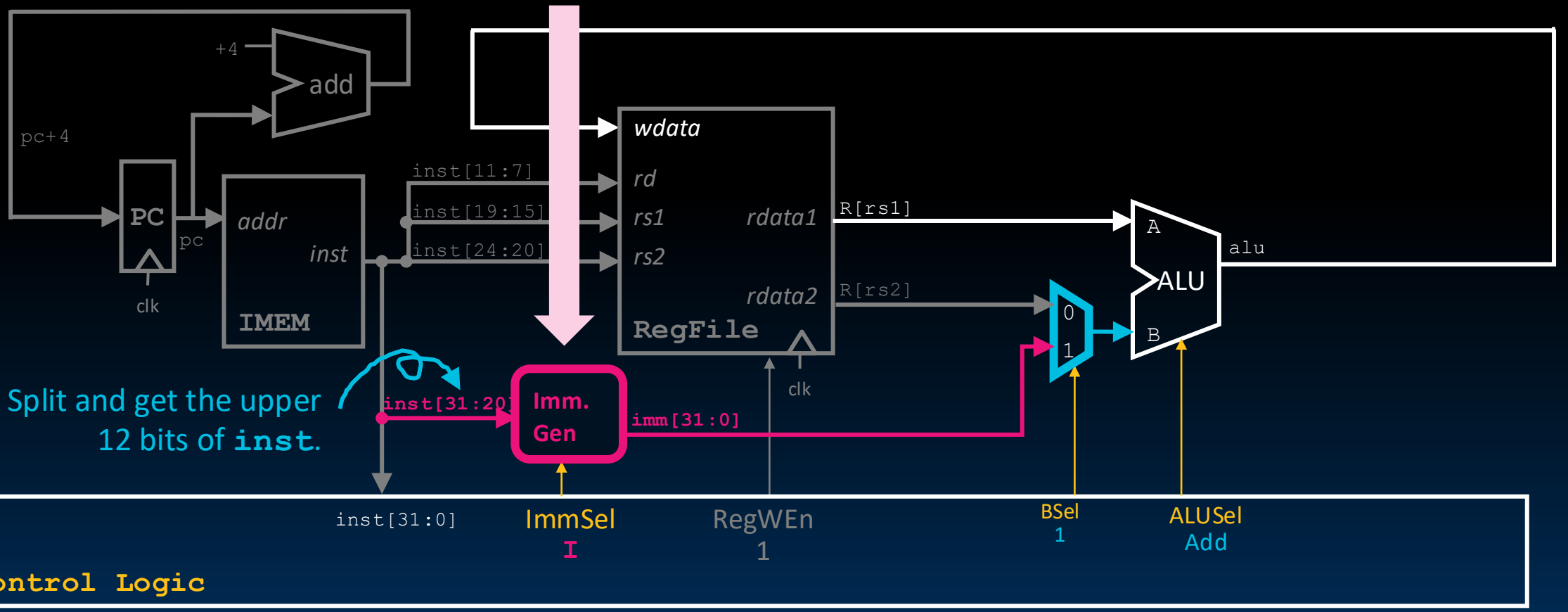
1. Control line BSel=1 selects the generated immediate **imm** for ALU input B.



2. New Block to Generate 32-bit Immediate

2. Immediate Generation Block builds a 32-bit immediate **imm** from instruction bits.

1. Control line BSel=1 selects the generated immediate **imm** for ALU input B.



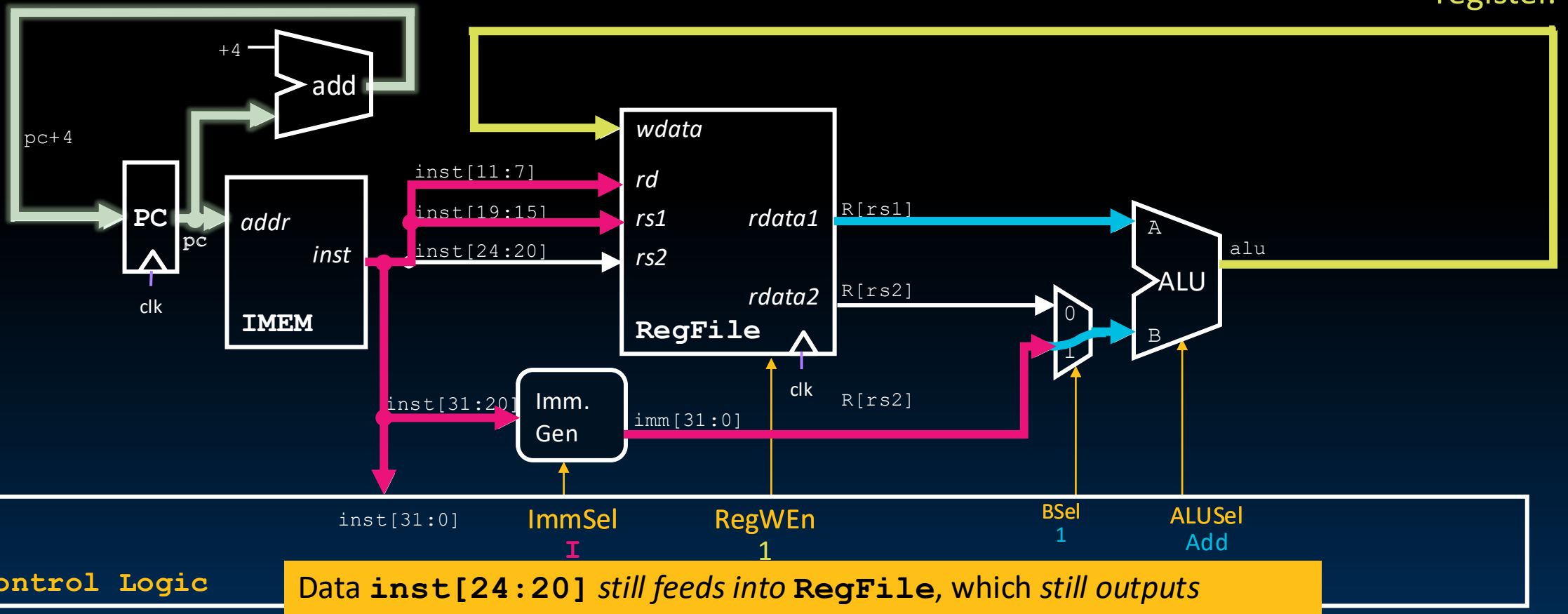
Tracing/Lighting the `addi` Datapath

Increment PC to next instruction.

Immediate Generation Block builds a 32-bit immediate **`imm`** from instruction bits.

Control line BSel=1 selects the generated immediate **`imm`** for ALU input B.

Write ALU output to destination register.



Data **`inst[24:20]`** *still feeds into* **`RegFile`**, which *still outputs* **`R[rs2]`**.
However, control **`Bsel=1`** means **`R[rs2]`** **data line is *ignored***.

Supporting Loads & Stores

- 5 Steps to a RISC-V Instruction
- R-Type: **`add`** datapath
- R-Type: **`sub`** datapath
- I-Type: **`addi`** datapath
- Supporting Loads and Stores
- Designing the Immediate Generation Block
- [Reference] CPU Elements

Implementing the `lw` Instruction

- `lw` uses I-Type: `lw rd imm(rs1)`

- **Similar datapath to `addi`, but creates an address (not the final value stored).**

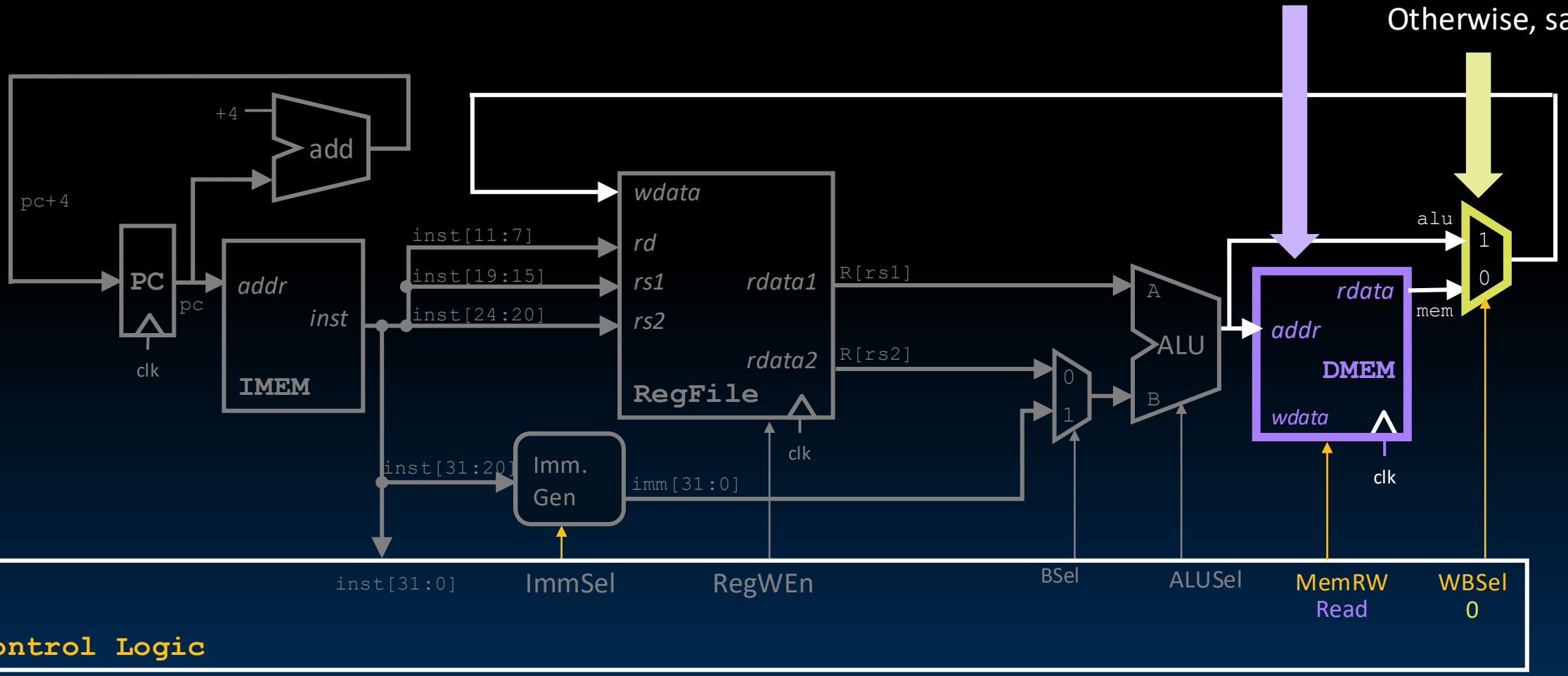
  `addr` = (Base register `rs1`) + (sign-extended `imm` offset)

- **State element access now includes a memory read!**
  - DMEM (read word at address `addr`)
  - RegFile `R[rs1] # read; R[rd] # write`
  - PC `PC = PC + 4`

Saving Memory Read to RegFile

Read memory at address **`alu = R[rs1] + imm`**.

If load instruction, save **`mem`**. Otherwise, save **`alu`**.



Lighting the LOAD Datapath

Load uses all five stages of the datapath!

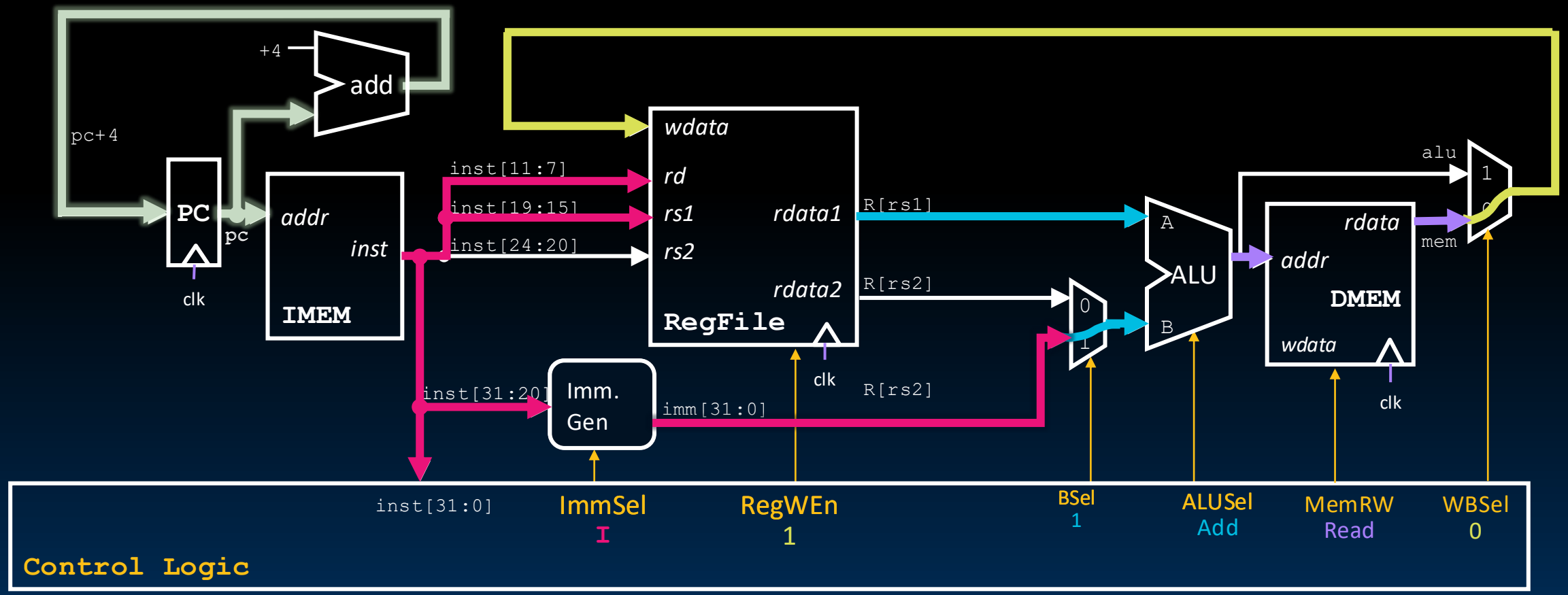
Increment PC to next instruction.

Immediate Generation Block builds a 32-bit immediate **imm**.

ALU computes address **alu = R[rs1] + imm**.

Read memory at address **alu**.

Write loaded memory value **mem** to register.



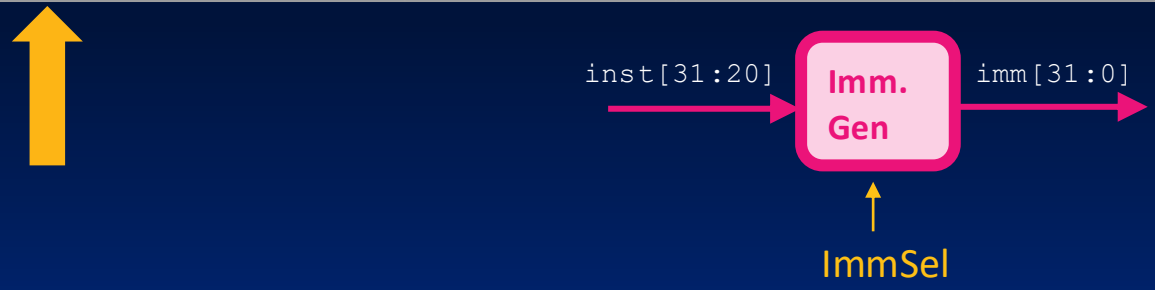
Designing the Immediate Generation Block

- 5 Steps to a RISC-V Instruction
- R-Type: **`add`** datapath
- R-Type: **`sub`** datapath
- I-Type: **`addi`** datapath
- Supporting Loads and Stores
- Designing the Immediate Generation Block
- [Reference] CPU Elements

[Review] Swirl Immediate Bits around for HW Design

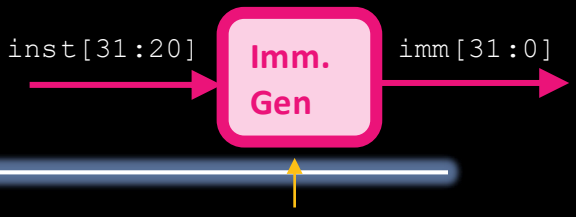
- Recall RISC-V keeps read/write register fields consistent across instruction formats.
- RISC-V also tries to keep **bit positions of immediates** consistent:

	31 26	24 20	19 15	14 12	11 7	6 0
I-Type	imm[11:0]		rs1	funct3	rd	opcode
	imm[11:5]	imm[4:0]				
	yxxxxxx	wwwwv	10011			
S-Type	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
	yxxxxxx				wwwwv	
B-Type	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
	zxxxxxx				wwwwy	

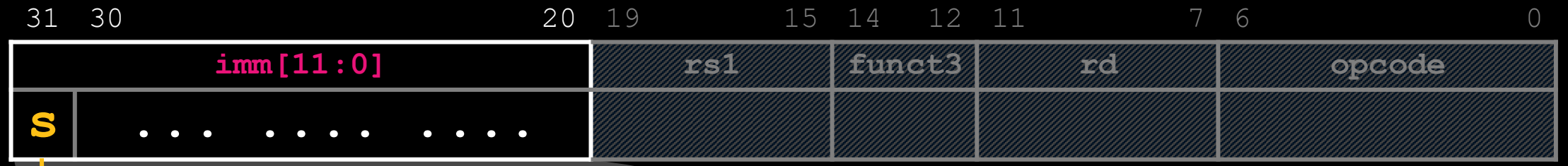


I-Type Immediate

GOT TO HERE



Instruction
`inst[31:0]`

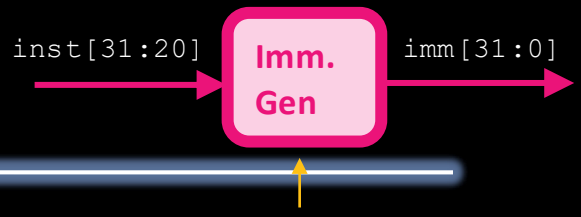


Copy *upper* 12 bits of instruction, **`inst[31:20]`**, to *lower* 12 bits of immediate, **`imm[11:0]`**.

Immediate
`imm[31:0]`

Sign-extend: Copy **`inst[31]`** to upper 20 bits of immediate, **`imm[31:12]`**.

S-Type Immediate (1/2)



Instruction `inst[31:0]`

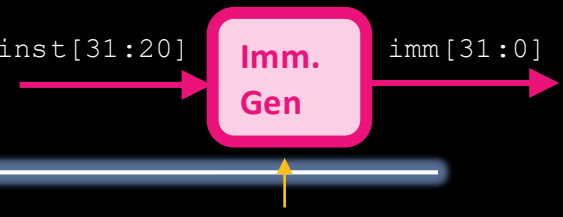
	31 30 25	24 20	19 15	14 12	11 7	6 0
I-Type	`imm[11|10:5]`	`imm[4:0]`	`rs1`	`funct3`	`rd`	`I-OPCODE`
S-Type	`imm[11|10:5]`	`rs2`	`rs1`	`funct3`	`imm[4:0]`	`S-OPCODE`

Sign-extend **`instr[31]`** to **`imm[31:12]`**.

For now, wire **`instr[31]`** directly to **`imm[11]`** (more next slide).

Immediate `imm[31:0]`

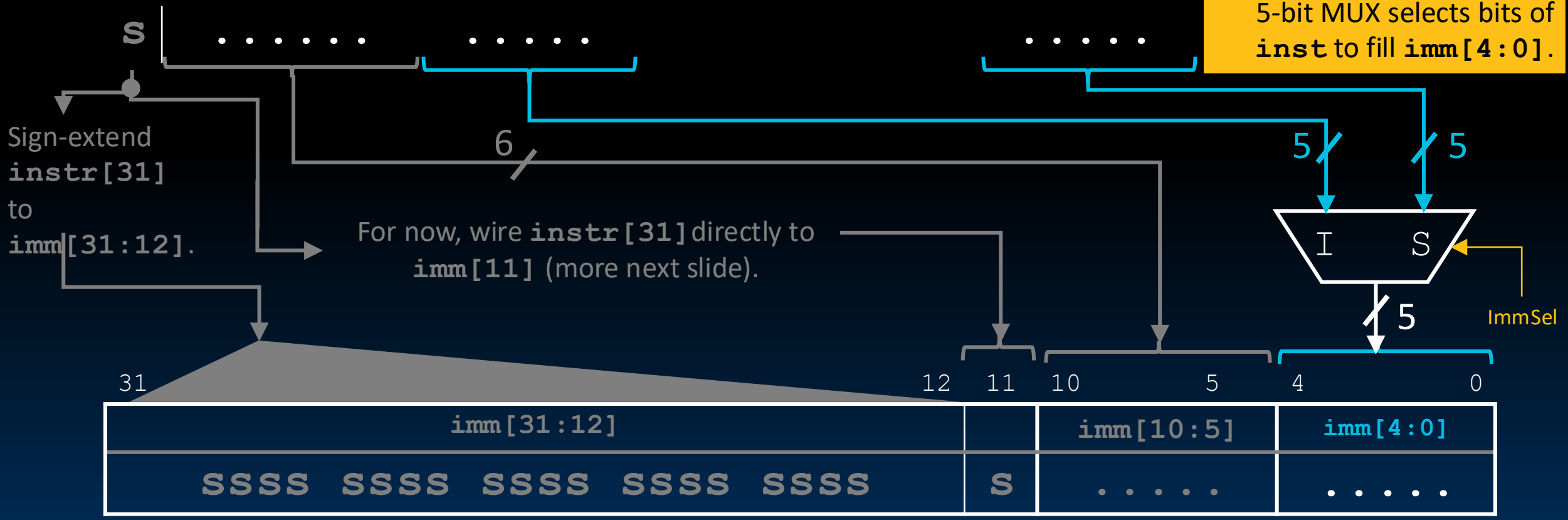
S-Type Immediate (2/2)



Instruction `inst[31:0]`

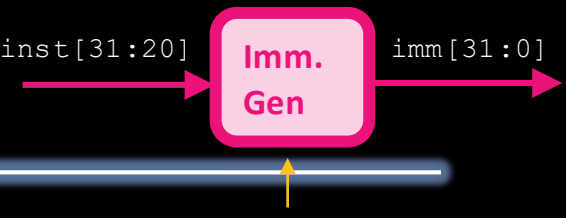
	31 30 25	24 20	19 15	14 12	11 7	6 0
I-Type	`imm[11|10:5]`	`imm[4:0]`	`rs1`	`funct3`	`rd`	`I-OPCODE`
S-Type	`imm[11|10:5]`	`rs2`	`rs1`	`funct3`	`imm[4:0]`	`S-OPCODE`

5-bit MUX selects bits of **`inst`** to fill **`imm[4:0]`**.

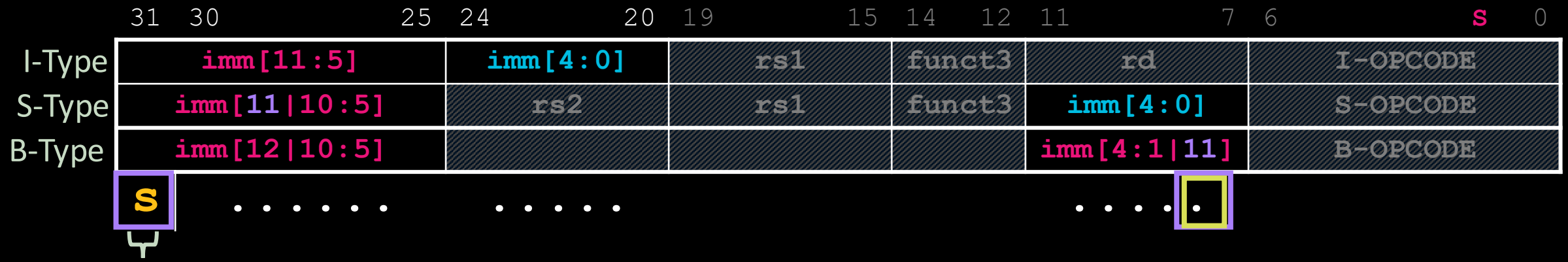


Immediate `imm[31:0]`

B-Type Immediates, etc.



Instruction `inst[31:0]`



`instr[31]` is always the sign bit.

MUX for **`imm[11]`**:

- S: **`instr[31]`**
- B: **`instr[7]`**

MUX for **`imm[0]`**:

- S: **`instr[7]`**
- B: **`0`** (implicit 0; half-words → bytes)

Immediate `imm[31:0]`

Read the notes!!

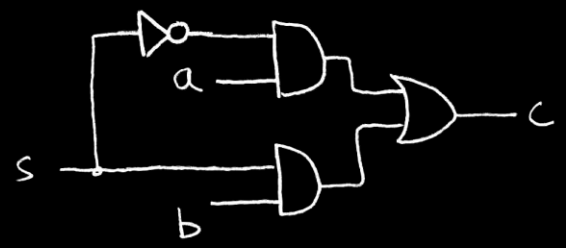
[Reference] CPU Elements

- 5 Steps to a RISC-V Instruction
- R-Type: **`add`** datapath
- R-Type: **`sub`** datapath
- I-Type: **`addi`** datapath
- Supporting Loads and Stores
- Designing the Immediate Generation Block
- [Reference] CPU Elements

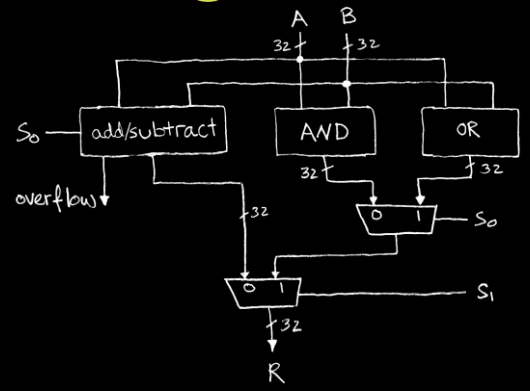
Combinational Logic Blocks, State Elements

- **Logic subcircuits as block diagrams:**

Last time

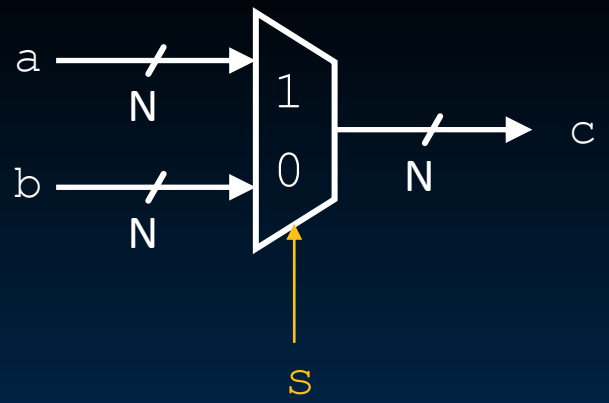


1-bit-wide 2-to-1MUX

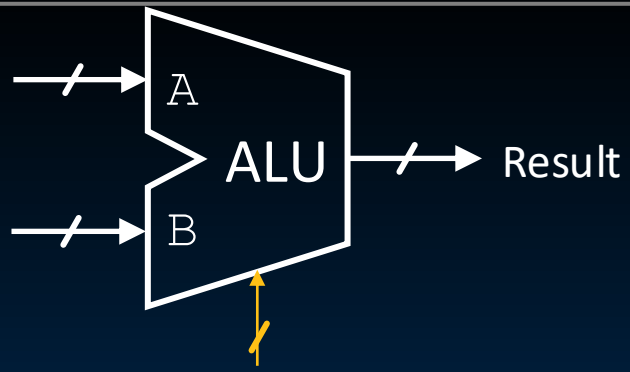


Simple ALU (add,sub,and,or)

This time

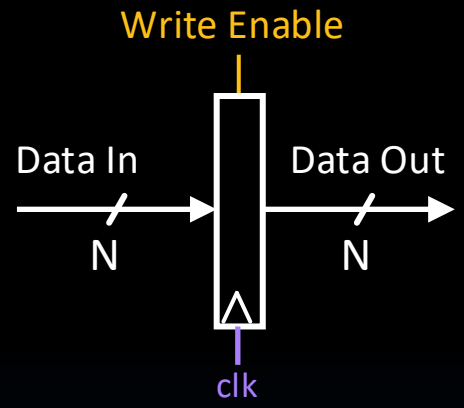


32-bit-wide 2-to-1 MUX



ALU (ALUOp selects from multiple operations)

- **State elements:**

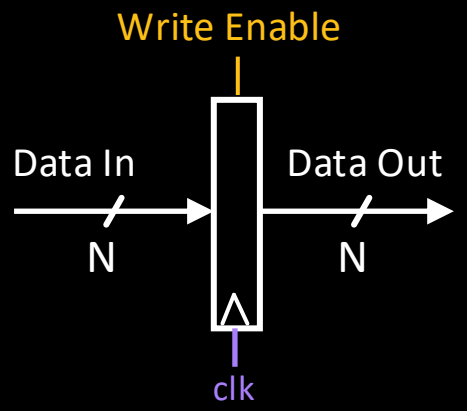


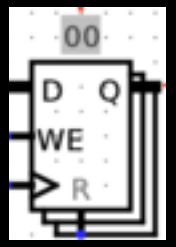
Within a clock cycle:

- Combinational logic updates occur after delay through the block.
- State element updates occur on the **next** rising clock edge.

State Elements (1/3): Program Counter

Program Counter





A register in Logisim

The Program Counter is a 32-bit Register.

Register File Reg

Memory MEM

- Input:
  - N-bit data input bus
  - Write Enable "Control" bit (1: asserted/high, 0: deasserted/0)
- Output:
  - N-bit data output bus
- Behavior:
  - If Write Enable is 1 on *rising clock edge*, set Data Out=Data In.
  - *At all other times*, Data Out will not change; it will output its current value.

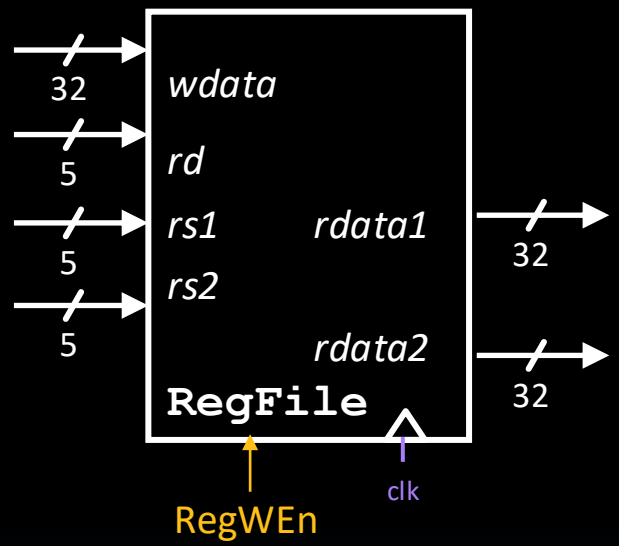
State Elements (2/3): Register File

Program Counter

PC

Register File Reg

RegFile



Register File (RegFile) has 32 registers.

- Input:
  - One 32-bit input data bus, *wdata*.
  - Three 5-bit select busses, *rs1*, *rs2*, and *rd*.
  - RegWEn control bit.
- Output:
  - Two 32-bit output data busses, *rdata1* and *rdata2*.

Memory MEM

IMEM

DMEM

- Registers are accessed via their 5-bit register numbers:
  - `R[rs1]`: *rs1* selects register to put on *rdata1* bus out.
  - `R[rs2]`: *rs2* selects register to put on *rdata2* bus out.
  - `R[rd]`: *rd* selects register to be written via *wdata* when RegWEn=1.
- Clock behavior: Write operation occurs on *rising clock edge*.
  - Clock input only a factor on write!
  - All read operations behave like a combinational block:
    - If *rs1*, *rs2* valid, then *rdata1*, *rdata2* valid after *access time*.

For read operations (RegWEn=1), RegFile behaves like a combinational block.

State Elements (3/3): Memory

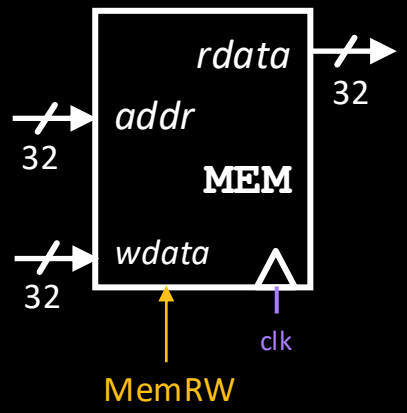
Program Counter PC

Register File RegFile

Memory MEM

IMEM

DMEM



Memory is "magic." For this class:

- 32-bit byte-addressed memory space; and
- Memory access with 32-bit words.

- Memory words are accessed as follows:
  - Read: Address *addr* selects word to put on *rdata* bus.
  - Write: Set MemRW=1.
    Address *addr* selects word to be written with *wdata* bus.
- Like RegFile, clock input is only a factor on write.
  - If MemRW=1, write occurs on rising clock edge.
  - If MemRW=0 and *addr* valid, then *rdata* valid after access time.

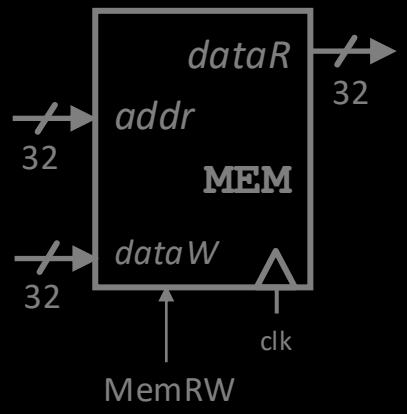
For read operations (MemRW=0), memory behaves like a combinational logic block.

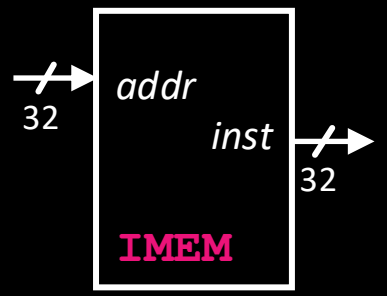
"Two" Memories: IMEM, DMEM

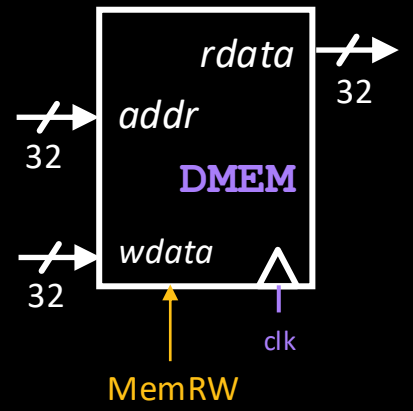
Program Counter

Register File Reg

Memory MEM







- Current abstraction: Memory holds both instructions and data in one contiguous 32-bit memory space.
- In our processor, we'll use two "separate" memories:
  - **IMEM**: A *read-only* memory for fetching instructions.
  - **DMEM**: A memory for loading (read) and storing (write) data words.
  - Under the hood, these are placeholders for L1 caches: **`L1i`**, **`L1d`**.
- IMEM is read-only and therefore always behaves like a combinational block.